

ASSESSMENT – 2 SOLUTION

1. Doctor Shift Scheduling using Backtracking Search

Problem Understanding

A hospital must assign three doctors (D1, D2, and D3) to three different shifts: Morning, Afternoon, and Night. The schedule must satisfy the following constraints:

- D1 cannot work the Night shift.
- D3 cannot work the Morning shift.
- D2 must be assigned before D3 (Morning < Afternoon < Night).
- Only one doctor can be assigned to each shift.
- Every doctor must receive exactly one shift.

This is a **Constraint Satisfaction Problem (CSP)** where the objective is to find a valid assignment that satisfies all the given constraints.

Method Selection

The problem is solved using the **Backtracking Search Algorithm**. Backtracking assigns shifts to doctors one by one while checking the constraints after each assignment. If any constraint is violated, the algorithm returns to the previous step and tries another possible assignment until a valid schedule is obtained.

Solution Process

Variables

- D1
- D2
- D3

Domain

- Morning
- Afternoon
- Night

Constraints

- D1 $\neq$ Night
- D3 $\neq$ Morning
- D2 before D3
- One doctor per shift
- Every doctor assigned exactly one shift

Assignment Steps

Step	Assignment	Status
1	D1 $\rightarrow$ Morning	✓ Valid
2	D2 $\rightarrow$ Afternoon	✓ Valid
3	D3 $\rightarrow$ Night	✓ Valid

Final Schedule

Doctor Shift

D1	Morning
D2	Afternoon
D3	Night

Interpretation of Result

The Backtracking Search algorithm successfully generated a valid duty schedule by satisfying all the given constraints. Invalid assignments were eliminated automatically through backtracking, resulting in the final schedule:

D1 $\rightarrow$ Morning

D2 $\rightarrow$ Afternoon

D3 $\rightarrow$ Night

2. Robot Grid Navigation using Breadth First Search (BFS)

Problem Understanding

A robot is placed in a **5 × 5 grid** and must travel from the **Start (S)** position to the **Goal (G)** position while avoiding blocked cells. The robot can move only in four directions (Up, Down, Left, Right), and each move has a cost of **1**. The Manhattan Distance heuristic is used to estimate the remaining distance to the goal. The objective is to determine the shortest path using **Breadth First Search (BFS)**.

Method Selection

Breadth First Search (BFS) is selected because it explores nodes level by level and guarantees the shortest path when all edge costs are equal. A queue is used to store the frontier nodes following the First-In First-Out (FIFO) principle.

Solution Process

Initial Position

Start = (0,0)

Goal = (4,4)

Manhattan Distance

Formula:

$$|x_1 - x_2| + |y_1 - y_2|$$

For the Start and Goal positions:

$$|4-0| + |4-0| = 8$$

BFS Expansion

Step Expanded Node

- 1 S
- 2 (0,1)
- 3 (1,0)
- 4 (2,0)

Step Expanded Node

5 (2,1)

6 (2,2)

7 (3,2)

8 (3,3)

9 (3,4)

10 Goal

Shortest Path

$S \rightarrow (1,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (3,2) \rightarrow (3,3) \rightarrow (3,4) \rightarrow \text{Goal}$

Total Cost

8

Interpretation of Result

Breadth First Search successfully found the shortest path from the Start node to the Goal node while avoiding obstacles. Since every movement had the same cost, BFS produced the optimal solution with a total path cost of **8**.

3. Autonomous Rescue Robot using Uniform Cost Search

Problem Understanding

An autonomous rescue robot operates in a disaster-affected area where obstacles and damaged roads may be present. The robot must identify the safest and least-cost path to reach victims while using available sensor information. The robot should behave as a rational intelligent agent and apply an appropriate search strategy to perform rescue operations efficiently.

Method Selection

The robot is modeled as an **Intelligent Agent** that observes the environment through sensors and performs actions using actuators. **Uniform Cost Search (UCS)** is chosen

because it always expands the node with the lowest cumulative path cost, ensuring the optimal path when all edge costs are non-negative.

Solution Process

Percepts

- Obstacle locations
- Victim locations
- Road conditions
- Battery level
- Sensor and camera data

Actions

- Move Forward
- Turn Left / Right
- Avoid Obstacles
- Rescue Victims
- Send Information

Environment Characteristics

Property	Type
Observability	Partially Observable
Environment	Dynamic
Nature	Stochastic
Decision Process	Sequential
Agents	Single Agent

Uniform Cost Search Steps

1. Insert the start node into the priority queue.
2. Expand the node with the lowest path cost.

3. Generate neighbouring nodes.
4. Update cumulative costs.
5. Repeat until the goal node is reached.
6. Return the least-cost path.

Advantages

- Produces the optimal path.
- Considers path cost instead of the number of steps.
- Suitable for rescue and navigation applications.
- Efficient for weighted graphs.

Interpretation of Result

The Autonomous Rescue Robot successfully performs rescue operations by acting as a rational intelligent agent. Using Uniform Cost Search, the robot selects the least-cost path, avoids obstacles, and reaches the target efficiently. This approach improves the effectiveness and reliability of rescue missions in dynamic environments.